

JSON Module

It looks like a dictionary but is just text (string format).

Commonly used in APIs, web development, config files, and data storage.

Example JSON string:

```
{  
  "name": "Neeraj",  
  "age": 25,  
  "skills": ["Python", "Django"]  
}
```

Purpose of `json` Module

The Python `json` module provides tools to:

1. **Convert Python objects → JSON string/file**

(`dump`, `dumps`)

2. **Convert JSON string/file → Python objects**

(`load`, `loads`)

Core Methods in `json` Module

1. `json.dump(obj, file)` → Write JSON to a file
2. `json.dumps(obj)` → Convert Python object → JSON string
3. `json.load(file)` → Read JSON from a file → Python object
4. `json.loads(string)` → Convert JSON string → Python object

`json.dumps()` return json as a string

`use` When you want to get JSON as a string (e.g., API response, printing, logging)

```
import json  
d = {  
    'name': 'Neeraj',  
    'age': 37,  
    'skill': ['HTML', 'css', 'python', 'Django']  
}  
print(json.dumps(d, indent=4))
```

Output

```
{
    "name": "Neeraj",
    "age": 37,
    "skill": [
        "HTML",
        "css",
        "python",
        "Django"
    ]
}
```

```
import json
```

```
d = {
    'name': 'Neeraj',
    'age': 37,
    'skill': ['HTML', 'css', 'python', 'Django']
}
print(json.dumps(d))
```

Output

```
{"name": "Neeraj", "age": 37, "skill": ["HTML", "css", "python", "Django"]}
```

`json.dump()` Writes JSON directly to a file.

`Use` When you want to save JSON to a file

```
import json
```

```
data = {"name": "Neeraj", "age": 25, "skills": ["Python", "Django"]}
```

```
with open("data.json", "w") as f:
    json.dump(data, f)
```

Output

create data.json file and write content on that file

```
{"name": "Neeraj", "age": 25, "skills": ["Python", "Django"]}
```

```
import json
```

```
data = {"name": "Neeraj", "age": 25, "skills": ["Python", "Django"]}
```

```
with open("data1.json", "w") as f:
```

```
json.dump(data, f, indent=4)
```

Output

create data.json file and write content on that file.

```
{
    "name": "Neeraj",
    "age": 25,
    "skills": [
        "Python",
        "Django"
    ]
}
```

json.dump() vs json.dumps()

Feature	<code>json.dump()</code>	<code>json.dumps()</code>
Use case	Serialize Python object and write JSON to a file	Serialize Python object and return JSON as a string
Input	Python object + file object	Python object
Output	Writes JSON to a file (returns <code>None</code>)	JSON string
Syntax	<code>json.dump(obj, file_object, indent=4)</code>	<code>json.dumps(obj, indent=4)</code>
When to use	When you need to save JSON data into a file	When you need a JSON string (e.g., API response, printing, logging)

`json.loads()` is a function in Python's `json` module that is used to **convert (deserialize) a JSON string into a Python objects like dict, list etc..**

```
import json
```

```
json_string = '{"name": "Neeraj", "age": 25, "skills": ["Python", "Django"]}'
python_obj = json.loads(json_string)
```

```
print(python_obj)
```

Output

```
{'name': 'Neeraj', 'age': 25, 'skills': ['Python', 'Django']}
<class 'dict'>
```

```
import json
```

```
json_string = '["apple", "banana", "cherry"]'
```

```
python_list = json.loads(json_string)
print(python_list)
print(type(python_list))
```

Output

```
['apple', 'banana', 'cherry']
<class 'list'>
```

`json.load()` is a function in Python's `json` module that is used to **read (deserialize) JSON data from a file and convert it into a Python object**.

It's like `json.loads()`, but instead of a **string**, it works with a **file object**.

Suppose we have a file `data.json`:

```
{
    "name": "Neeraj",
    "age": 25,
    "skills": ["Python", "Django"]
}
```

```
import json
```

```
with open("data.json", "r") as f:
    python_obj = json.load(f)
```

```
print(python_obj)
print(type(python_obj))
```

Output

```
{'name': 'Neeraj', 'age': 25, 'skills': ['Python', 'Django']}
<class 'dict'>
```

json.load() vs json.loads()

Feature	<code>json.load()</code>	<code>json.loads()</code>
Use case	Deserialize JSON from a file into a Python object	Deserialize JSON from a string into a Python object
Input	File object (opened in read mode) containing JSON	JSON string
Output	Python object (dict, list, etc.)	Python object (dict, list, etc.)
Syntax	<code>json.load(file_object)</code>	<code>json.loads(json_string)</code>

Feature	<code>json.load()</code>	<code>json.loads()</code>
When to use	When JSON data is stored in a file	When JSON data is received as a string (e.g., API response, user input)